

Programación imperativa

Tabla de contenidos

1	El conocimiento reside en el programa	3
1.1	Las fortalezas del modelo	4
1.2	El requisito fundamental	4
1.3	Cuando el algoritmo no es evidente	5

1 El conocimiento reside en el programa

Desde los primeros ordenadores hasta la actualidad, la inmensa mayoría del software se ha construido siguiendo un mismo principio fundamental.

El desarrollador describe explícitamente cómo debe resolverse un problema.

Es decir, un programa no contiene únicamente el objetivo que se desea alcanzar, si no que contiene también la secuencia exacta de pasos necesarios para alcanzarlo. Un lenguaje de tercera generación (3GL), como C, Java, C++, COBOL o Python, permite expresar esa secuencia mediante instrucciones perfectamente definidas y sin ambigüedad.

Un programa está formado por elementos como:

- Variables.
- Tipos de datos.
- Expresiones.
- Operadores.
- Condiciones.
- Bucles.
- Funciones y procedimientos.
- Clases y objetos.
- Estructuras de datos.

Estos elementos permiten construir algoritmos, los cuales no son mas que una secuencia ordenada de instrucciones que describe, paso a paso, cómo resolver un problema. Cada instrucción modifica el estado del sistema de una forma conocida y predecible. El procesador ejecuta exactamente las instrucciones escritas por el desarrollador, en el orden establecido por éste y siguiendo las reglas del lenguaje de programación.

Por ejemplo:

```
if (saldo >= importe) {  
    saldo -= importe;  
    realizarTransferencia();  
} else {  
    throw new SaldoInsuficienteException();  
}
```

En este ejemplo no existe ninguna interpretación posible.

Si la condición se cumple, se ejecutarán exactamente las instrucciones del primer bloque. Si no se cumple, se ejecutará el segundo. El resultado será siempre el mismo para una misma

entrada. La responsabilidad de decidir **qué hacer, cuándo hacerlo, cómo hacerlo y en qué orden hacerlo** recae completamente sobre el ingeniero de software. El ordenador no toma decisiones. Ejecuta las instrucciones que recibe.

1.1. Las fortalezas del modelo

Este paradigma ha demostrado durante más de medio siglo ser extraordinariamente eficaz. Permite construir sistemas que son:

- Deterministas.
- Reproducibles.
- Verificables.
- Depurables.
- Trazables.
- Mantenibles.

Gracias a ello se han desarrollado sistemas críticos como:

- Sistemas bancarios.
- Control del tráfico aéreo.
- Telecomunicaciones.
- Sistemas industriales.
- Satélites.
- Sistemas sanitarios.
- Administración pública.

Durante décadas, este modelo ha constituido la base de la Ingeniería del Software.

1.2. El requisito fundamental

Sin embargo, este enfoque tiene un requisito imprescindible: **El ingeniero debe conocer previamente el algoritmo que resuelve el problema..** Antes de escribir una sola línea de código debe ser capaz de responder preguntas como:

- ¿Qué pasos debo ejecutar?
- ¿En qué orden?
- ¿Qué decisiones deben tomarse?
- ¿Qué casos especiales existen?
- ¿Cómo debo gestionar los errores?
- ¿Qué información necesito en cada momento?

En otras palabras:

El conocimiento necesario para resolver el problema está contenido explícitamente en el programa.

El ordenador simplemente ejecuta ese conocimiento.

1.3. Cuando el algoritmo no es evidente

Existen, sin embargo, problemas para los que resulta extremadamente difícil escribir un algoritmo preciso.

Por ejemplo:

- Resumir un contrato de cien páginas.
- Explicar el significado de un poema.
- Traducir un documento técnico.
- Detectar el tono de una conversación.
- Responder preguntas sobre miles de documentos.
- Mantener una conversación natural.

Las personas realizamos estas tareas de manera cotidiana; sin embargo, describir mediante una secuencia exacta de instrucciones cómo llevarlas a cabo resulta extraordinariamente complejo. Durante décadas se intentó resolver este problema mediante reglas, gramáticas, árboles sintácticos, sistemas expertos y numerosas técnicas de Procesamiento del Lenguaje Natural que, aunque obtuvieron buenos resultados en ámbitos concretos, ninguna consiguió ofrecer una solución general.

Es precisamente en este contexto donde aparecen los **Large Language Models (LLM)**. Su importancia no reside únicamente en que producen texto, su verdadera aportación consiste en introducir una forma completamente diferente de abordar problemas para los que no conocemos, o resulta inviable describir, un algoritmo explícito.

Comprender este cambio de paradigma será el objetivo de los siguientes capítulos.